Lecture 5: Unsupervised learning & dimension reduction

Alex Reinhart

Machine Learning II: 46-927

February 22–24, 2021

Announcements

- Project proposals reviewed on Gradescope; I left comments on some submissions
- Project presentations are on Tuesday, March 16 from 10am to 1pm; groups will be split into two different Zoom sessions
- Homework 4 (due Wednesday) is unintentionally long because of my mistake. I did not provide enough R example code, since you have done less R in prior courses than I thought.
  - There are 100 points of questions (6–8 points each)
  - When we transfer the grades to Canvas, we will make the assignment out of 78 points
  - You can either skip 22 points of questions or get bonus
- Remaining topics this mini: matrix factorization and deep learning
- Only 2 homeworks left, i.e. the last homework will be HW 6

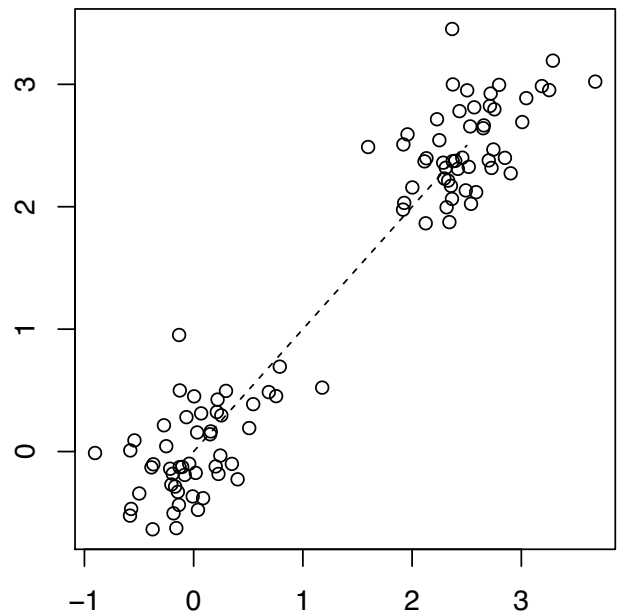
Today:

- Dimension reduction
  - PCA carefully
  - Non-linear dimension reduction
- Mixture models, matrix factorization

Principal component analysis

Principal component analysis (PCA). You saw this in Data Science and briefly earlier this term. I just want to remind you of a few things so that we have something to build on.

We are given a data matrix $X \in \mathbb{R}^{n \times p}$, meaning that we have n observations (row vectors) and p features (column vectors). We assume that the columns of X have been centered.



First principal component direction and score

The first principal component direction of X is the unit vector $v_1 \in \mathbb{R}^p$ that maximizes the sample variance of $Xv_1 \in \mathbb{R}^n$ when compared to all other unit vectors

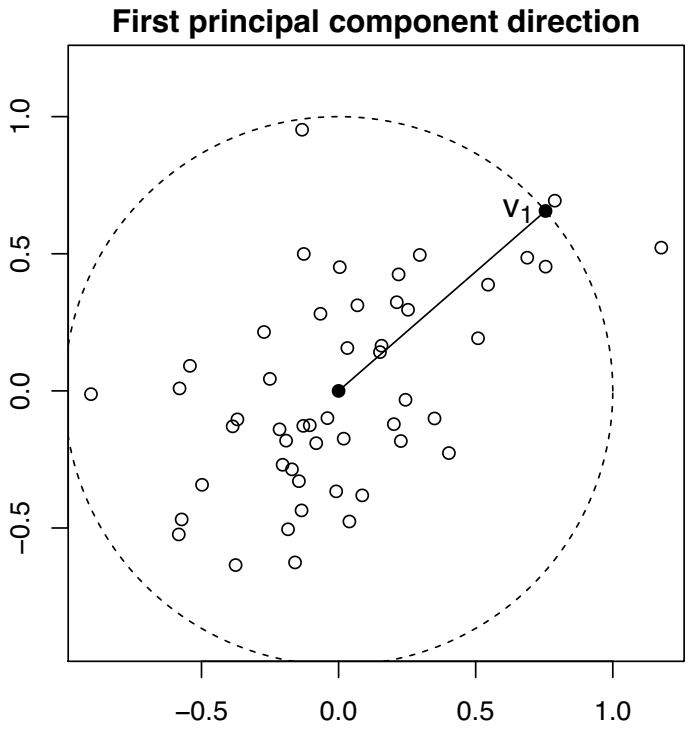
For any $v \in \mathbb{R}^p$, the vector $Xv \in \mathbb{R}^n$ has sample mean zero and sample variance $\frac{1}{n}(Xv)^T(Xv)$ (recall that we column centered X). Hence the first principal component direction $v_1 \in \mathbb{R}^p$ is

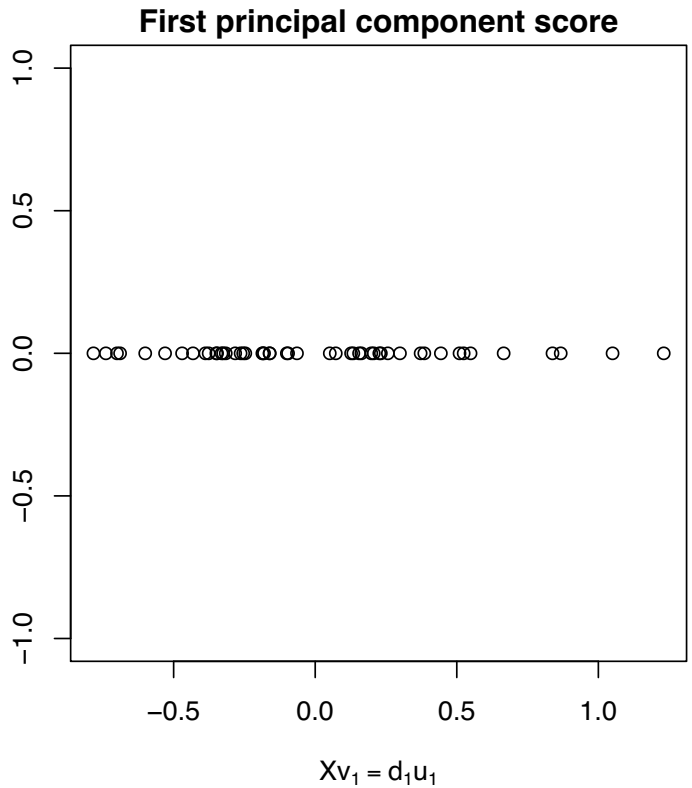
$$v_1 = \underset{\|v\|_2=1}{\operatorname{argmax}} \, (Xv)^T(Xv)$$

The vector $Xv_1 \in \mathbb{R}^n$ is called the first principal component score of X, and $u_1 = (Xv_1)/d_1 \in \mathbb{R}^n$ is the normalized first principal component score. Here $d_1 = \sqrt{(Xv_1)^T(Xv_1)}$, and d_1^2/n is the amount of variance explained by v_1

Example: first principal component direction and score

Same example data as earlier: $X \in \mathbb{R}^{50 \times 2}$





Why does this agree with our eigenvalue interpretation?

$$X^T X = U D U^T$$

U: orthonormal

D: diagonal

$$\underset{\|v\|_2 = 1}{\operatorname{argmax}} \; (Xv)^T Xv$$

$$v^T X^T X v$$

$$v^T U D U^T v$$

$$(U^T v)^T \begin{pmatrix} \lambda_1 & & 0 \\ & \lambda_2 & \\ 0 & & \ddots \; \lambda_p \end{pmatrix} \underbrace{(U^T v)}_{z}$$

Maximizing $z = \begin{pmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}$

Further principal component directions and scores

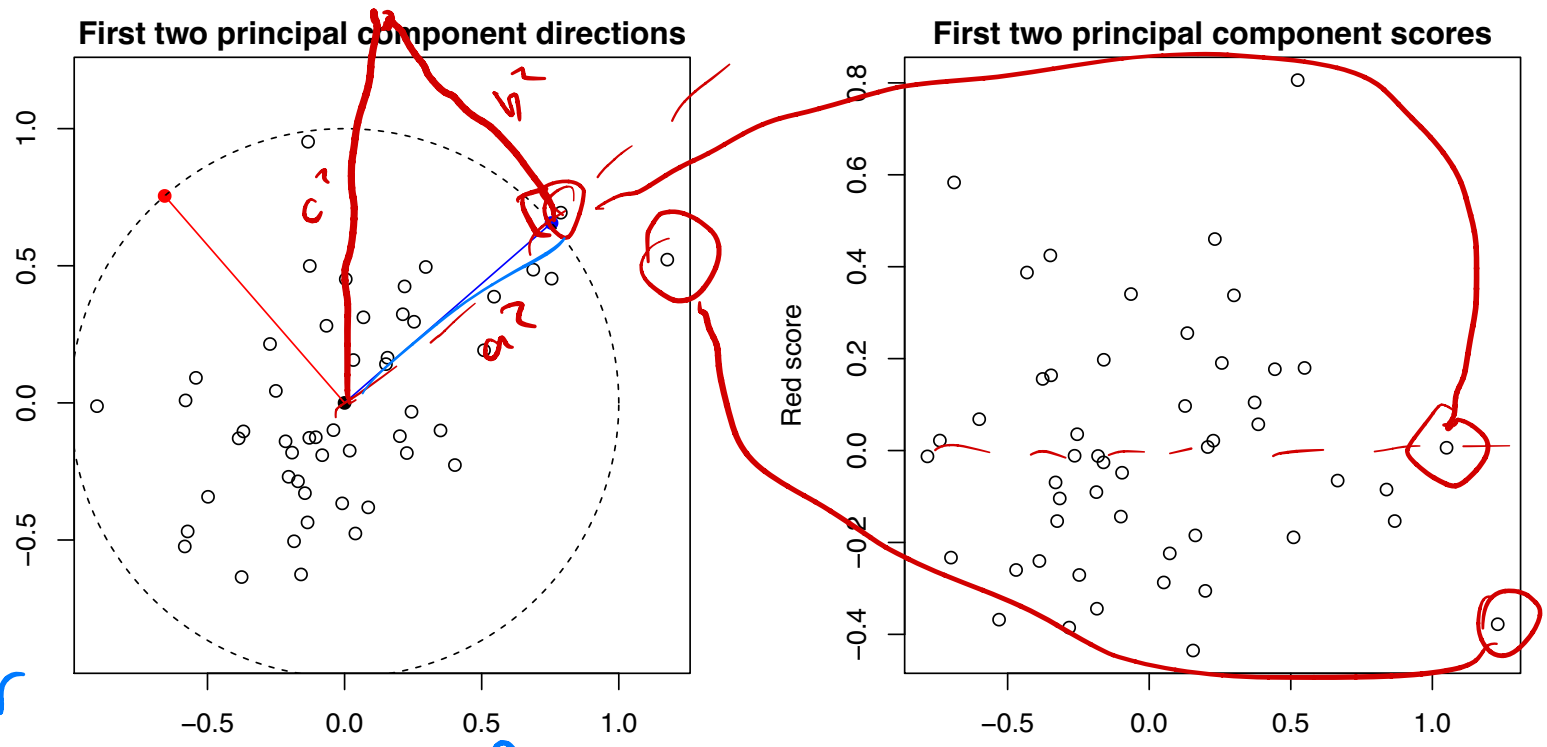
Given the $k-1$ principal component directions $v_1, \ldots, v_{k-1} \in \mathbb{R}^p$ (note that these are orthonormal), we define the kth principal component direction $v_k \in \mathbb{R}^p$ to be

$$v_k = \underset{\substack{\|v\|_2=1 \\ v^T v_j=0,\, j=1,\ldots,k-1}}{\operatorname{argmax}} (Xv)^T(Xv)$$

The vector $Xv_k \in \mathbb{R}^n$ is called the kth principal component score of X, and $u_k = (Xv_k)/d_k \in \mathbb{R}^n$ is the normalized kth principal component score. Here $d_k = \sqrt{(Xv_k)^T(Xv_k)}$, and d_k^2/n is the amount of variance explained by v_k

Example: second principal component direction and score

Same example data as earlier: $X \in \mathbb{R}^{50 \times 2}$



tot. var

$$\frac{1}{n}\sum_{i=1}^{n} x_i^T x_i = \frac{1}{n}\sum_{i=1}^{n}\left(x_{i1}^2 + x_{i2}^2\right) = \frac{1}{n}\sum_{i=1}^{n} x_{i1}^2 + \frac{1}{n}\sum_{i=1}^{n} x_{i2}^2$$

Properties and representations

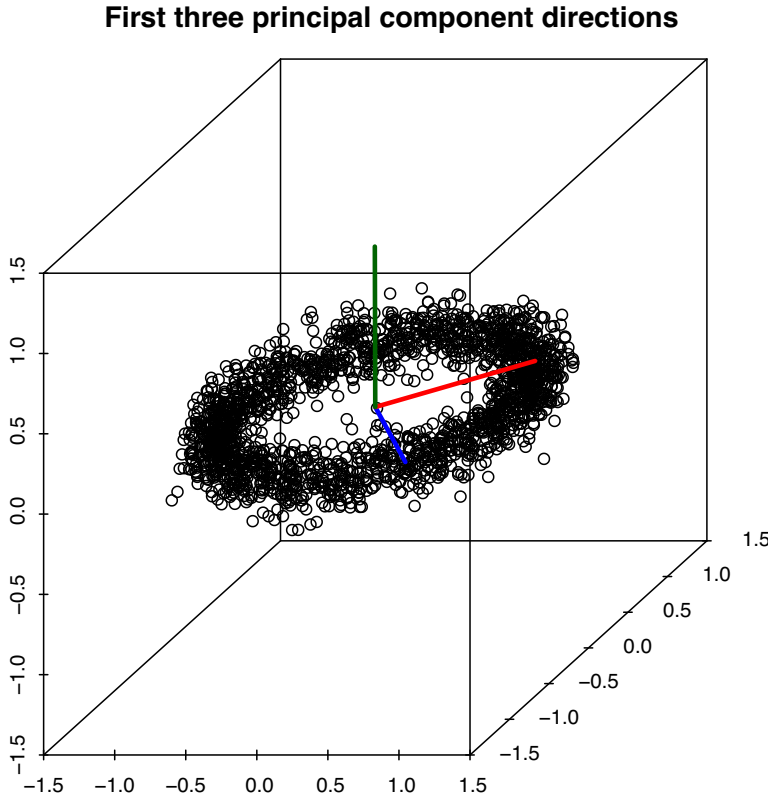
- For the kth principal component direction $v_k \in \mathbb{R}^p$ and score $u_k \in \mathbb{R}^n$, the entries of $Xv_k = d_k u_k$ are the scores from projecting X onto v_k, and the rows of $Xv_k v_k^T = d_k u_k v_k^T$ are the projected vectors

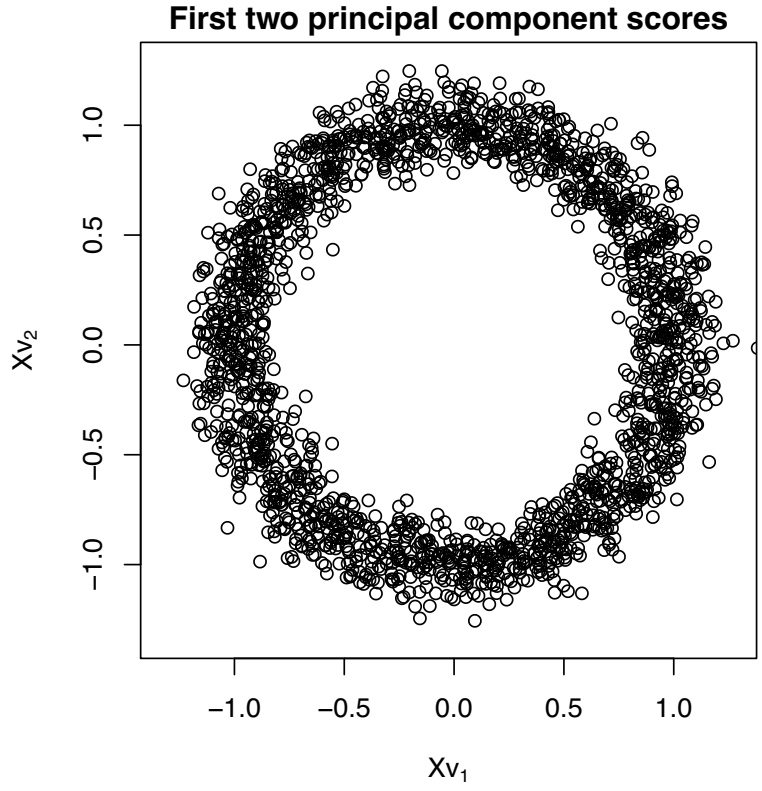
- The directions v_k and normalized scores u_k are only unique up to sign flips

- Concise representation: let the columns of $V \in \mathbb{R}^{p \times p}$ be the directions. Scores: columns of $XV \in \mathbb{R}^{n \times p}$. Projections onto V_k (first k columns of V): rows of $XV_k V_k^T \in \mathbb{R}^{n \times p}$

Example: principal component analysis in $\mathbb{R}^3$

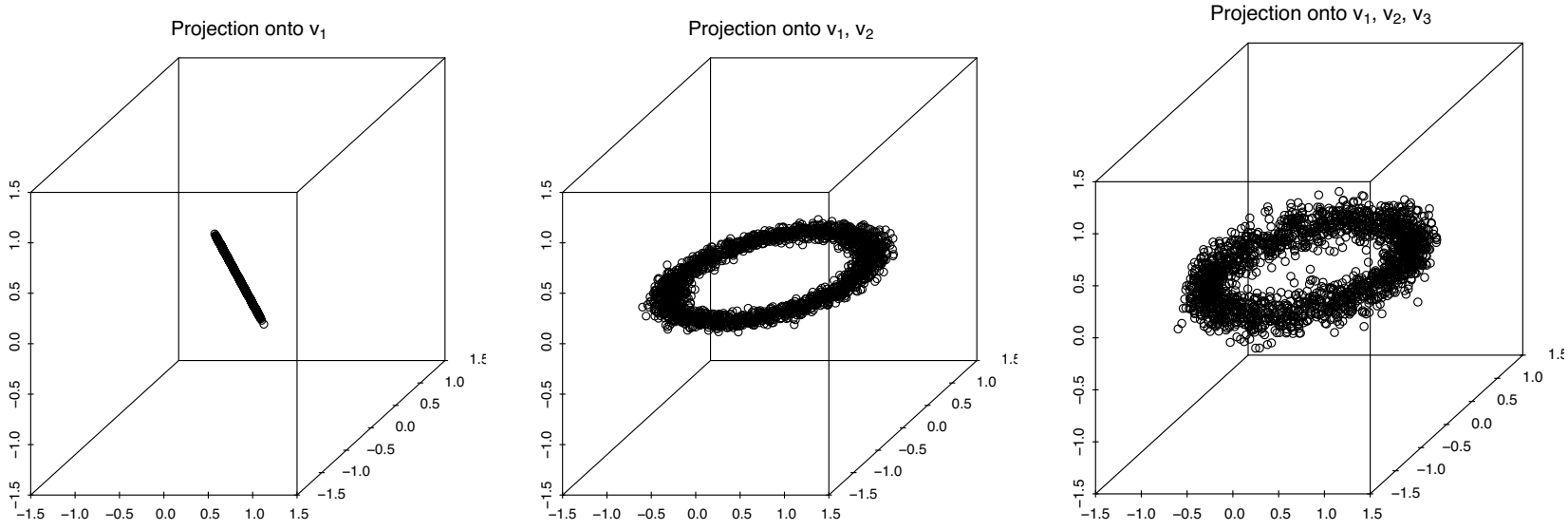
Example: $X \in \mathbb{R}^{2000 \times 3}$. Shown are the three principal component directions $v_1, v_2, v_3 \in \mathbb{R}^3$, and the scores from projecting onto the first two directions





Example: projecting onto principal component directions

Same example: $X \in \mathbb{R}^{2000\times 3}$, $v_1, v_2, v_3 \in \mathbb{R}^3$. What happens if replace X by its projection onto v_1? Onto v_1, v_2? Onto v_1, v_2, v_3?



The third plot looks exactly the same as the original data...

Proportion of variance explained

Recall that we said: d_k^2/n is the amount of variance explained by the kth principal component direction v_k

Pythagoras

Two facts:

- The total sample variance of X is $\frac{1}{n}\sum_{j=1}^{p} d_j^2$
- The total sample variance of $XV_kV_k^T$ is $\frac{1}{n}\sum_{j=1}^{k} d_j^2$ (amount of variance explained by $v_1, \ldots, v_k$)

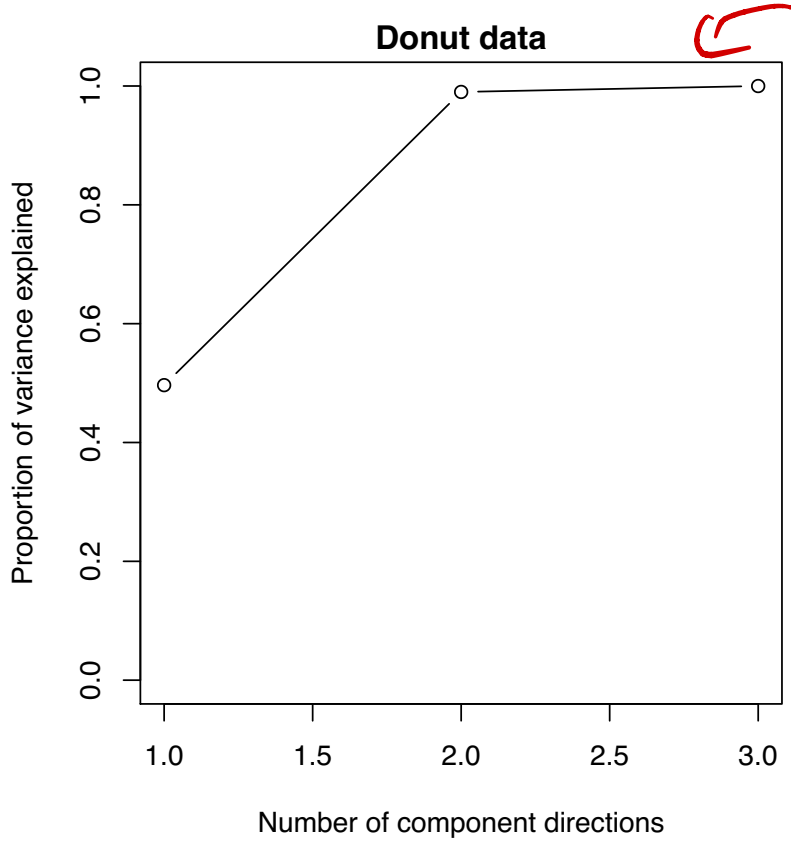
Hence the proportion of variance explained by the first k principal component directions $v_1, \ldots, v_k$ is

$$\frac{\sum_{j=1}^{k} d_j^2}{\sum_{j=1}^{p} d_j^2}$$

If this is high for a small value of k, then it means that the main structure in X can be explained by a small number of directions

Example: proportion of variance explained

Example: proportion of variance explained as a function of k, for the donut data



scree plot

Example: principal component analysis in $\mathbb{R}^9$

Example: data on glass composition from crime scenes. The data studies how 9 chemical measurements correspond to the type of glass.

Measurements: `Na, Mg, Al, Si, K, Ca, Ba, Fe`

Glass types: `building_float building_nonfloat containers headlamps tableware vehicle_float`

The data set contains measurements on 214 samples.

Dimension reduction via the principal component scores

As we've seen in the examples, dimension reduction via principal component analysis can be achieved by taking the first k principal component scores $Xv_1, \ldots, Xv_k \in \mathbb{R}^n$

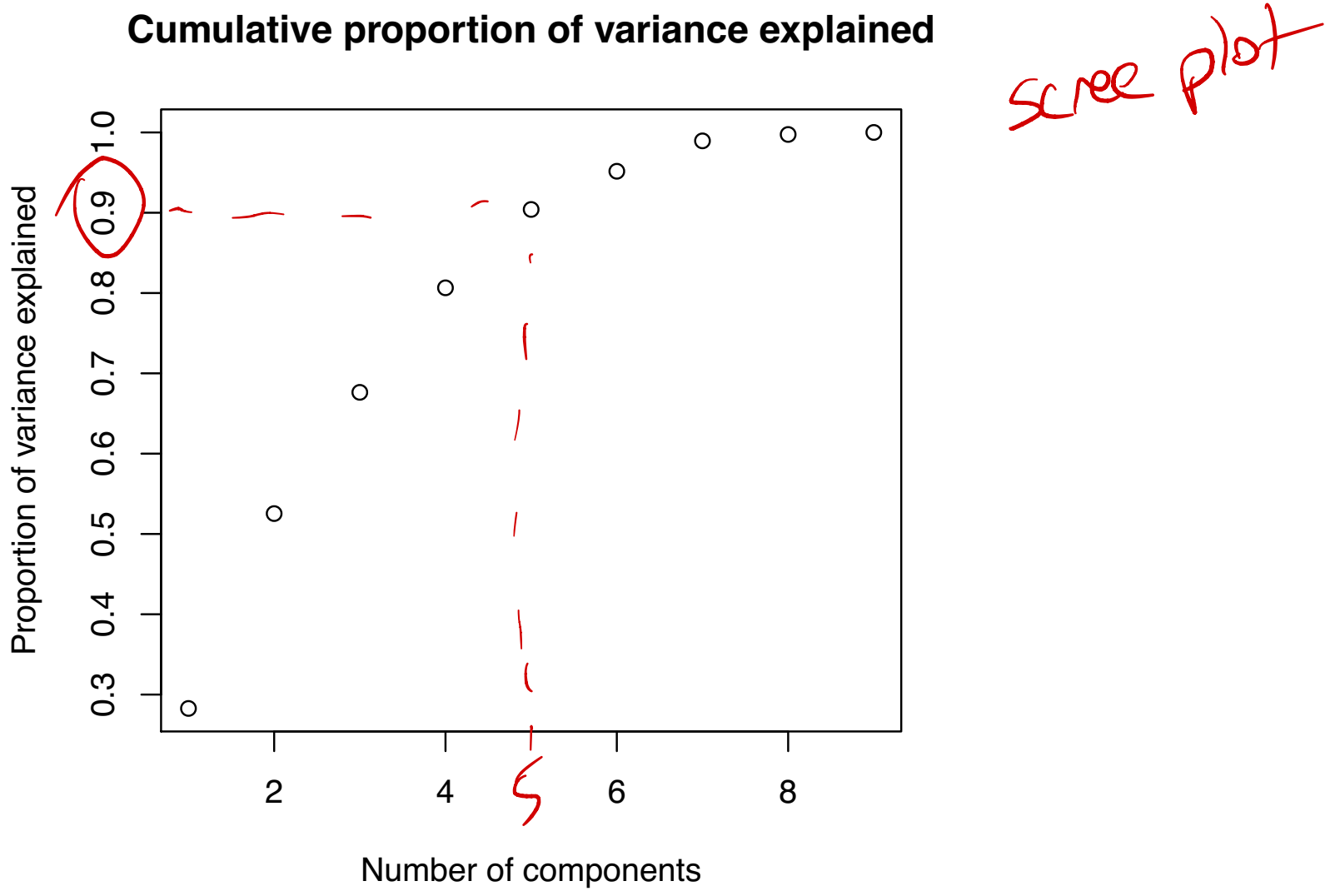
We can think of $Xv_1, \ldots, Xv_k$ as our new feature vectors, which is a big savings if $k \ll p$ (e.g. $k = 2$ or 3)

An important question: how good are these features at capturing the structure of our old features? Broken up into two questions:

1. How good are they, for a fixed k?
2. What exactly do we gain by increasing k?

Recall that the second question can be addressed by looking at the proportion of variance explained as a function of k

Example: proportion of variance explained, glass data



Approximation by projection

As for the first question, think about approximating X by $XV_kV_k^T$, the projection of X onto the first k principal component directions

An important alternate characterization of the principal component directions: given centered $X \in \mathbb{R}^{n\times p}$, if $V_k = [v_1 \ \ldots v_k] \in \mathbb{R}^{p\times k}$ is the matrix whose columns contain the first k principal component directions of X, then

$$XV_kV_k^T = \underset{\text{rank}(A)=k}{\text{argmin}} \ \|X - A\|_F^2 = \underset{\text{rank}(A)=k}{\text{argmin}} \sum_{i=1}^{n}\sum_{j=1}^{p}(X_{ij} - A_{ij})^2$$

Frobenius norm

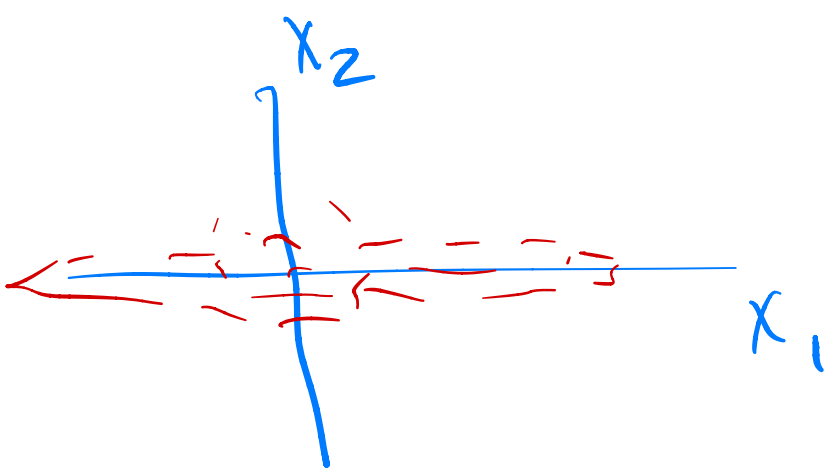
In other words, $XV_kV_k^T$ is the best rank k approximation to X

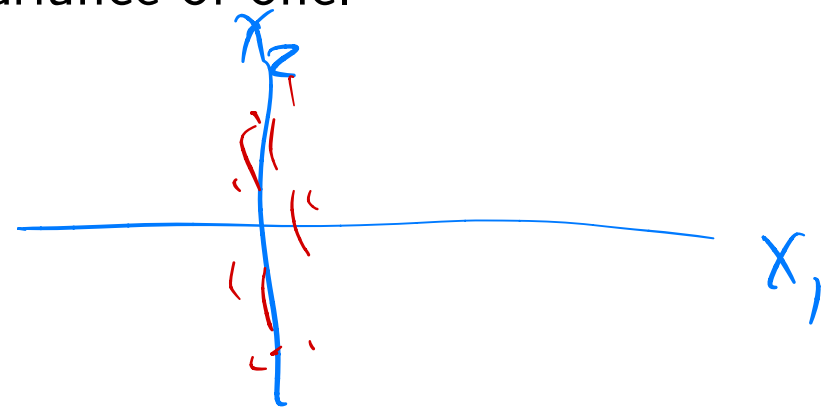
(Aside: the above problem is nonconvex, and would be very hard to solve in general!)

Scaling the features

We always center the columns of X before computing the principal component directions.

Another common pre-processing step is to scale the columns of X, i.e., to divide each feature by its sample variance, so that each feature in our new X has a sample variance of one.





Computing principal component directions

You know one way to compute the Principal component directions:

Eigenvalue Decomposition: We write $X^T X = V \Lambda V^T$, where the columns of V are the eigenvectors and Λ is the diagonal matrix of eigenvalues. Then

- The columns of V, v_j are the principal component directions.
- The elements λ_j / n are the amounts of variation explained.
- We can compute the scores $X v_j$.

It is worth seeing one "other" way to think of computing PCA.

Computing principal component directions: SVD

Singular value decomposition (SVD) of X:

$$\underset{n \times p}{X} = \underset{n \times p}{U} \underset{p \times p}{D} \underset{p \times p}{V^T}$$

Here $D = \mathrm{diag}(d_1, \ldots, d_p)$ is diagonal with $d_1 \geq \cdots \geq d_p \geq 0$, and U, V both have orthonormal columns. This gives us everything:

- columns of V, $v_1, \ldots, v_p \in \mathbb{R}^p$, are the principal component directions
- columns of U, $u_1, \ldots, u_p \in \mathbb{R}^n$, are the normalized principal component scores
- squaring the jth diagonal element of D and dividing by n, d_j^2/n, gives the variance explained by v_j

(Don't forget that we must first center the columns of X!)

Note that

$$X^T X = V D^2 V^T$$

and so $v_1, \ldots, v_p$ are eigenvectors of $X^T X$.

Note also that

$$XV = UDV^T V = UD$$

because $V^T V = I$. This means that

$$Xv_j = d_j u_j, \quad j = 1, \ldots, p$$

so that the u_j correspond to the normalized scores.

We will come back to the SVD later this lecture.

PCA in high dimensions

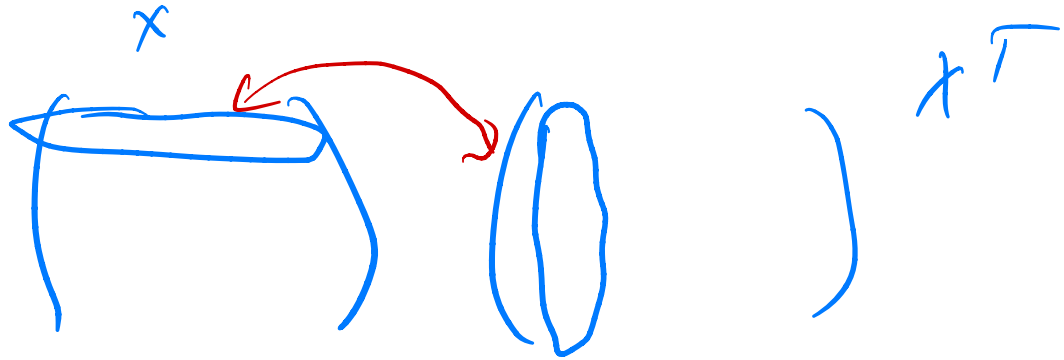
Imagine drawing $x \sim N(0, \Sigma)$, constructing a matrix $X \in \mathbb{R}^{n \times p}$, and running PCA.

PCA finds dimensions which capture most of the variation of X. We often think of this as reflecting dimensions which capture most of the "population" variance Σ.

In high dimensions, this will not tend to be the case. There will be so much variation in the realization of $X^T X$ that the best directions for X will not be the best directions for Σ.

This also means that the directions that are good for X will not tend to be nearly as good for a new sample X'.

The inner-product matrix



Given $X \in \mathbb{R}^{n \times p}$, the matrix $XX^T \in \mathbb{R}^{n \times n}$ is the inner-product matrix. If X has ith row $x_i \in \mathbb{R}^p$, then $(XX^T)_{ij} = x_i^T x_j$

Suppose that we wanted the principal component scores of X, but we only had XX^T. Could we still compute them?

The inner-product matrix

Yes! Using the singular value decomposition $X = UDV^T$, we have

$$XX^T = UD^2U^T$$

This is called an eigendecomposition of XX^T because the columns of U are eigenvectors of XX^T.

Hence we can compute the eigendecomposition or "factorize" the inner product matrix XX^T, and then the scores are given by the columns of UD, i.e., $d_j u_j$, $j = 1, \ldots, p$

Low-dimensional representation from distances only

Suppose that instead of measuring $X \in \mathbb{R}^{n \times p}$ directly, we only measured the distances between pairs of observations,

$$\Delta_{ij} = \|x_i - x_j\|_2, \quad i, j = 1, \ldots, n$$

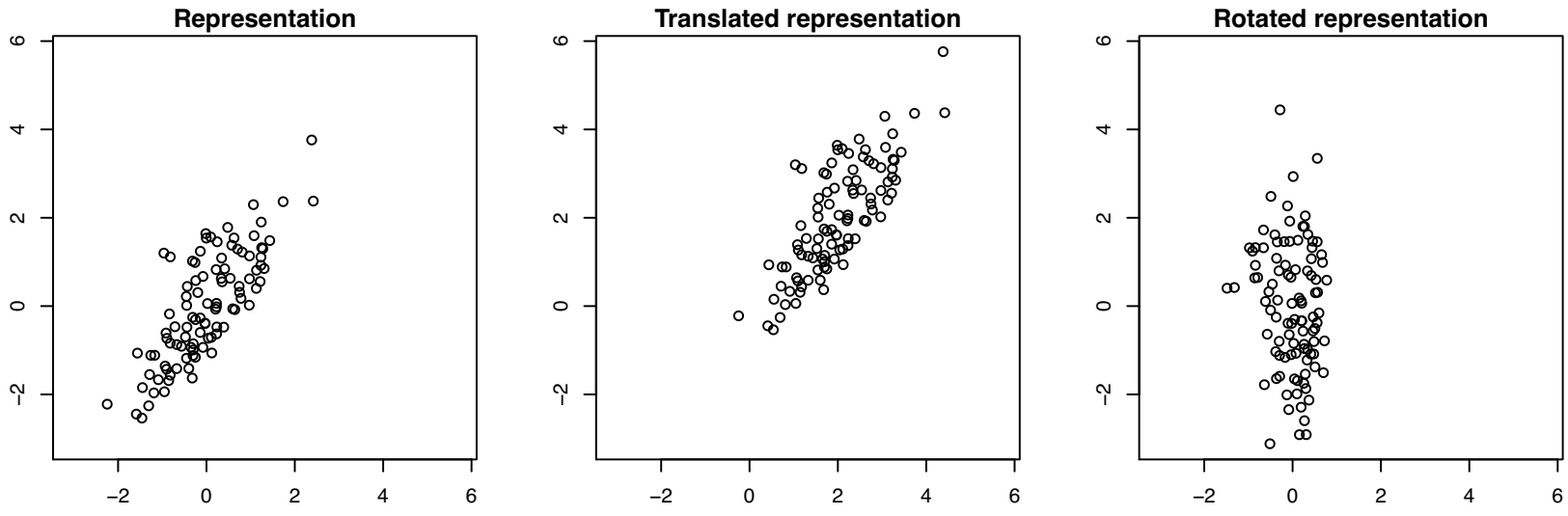
This gives us a distance matrix $\Delta \in \mathbb{R}^{n \times n}$

We want a lower-dimensional representation $z_1, \ldots, z_n \in \mathbb{R}^k$, for some small k (e.g. $k = 2$ or 3), such that $\|z_i - z_j\|_2 \approx \Delta_{ij}$, for every $i, j = 1, \ldots, n$

We saw that the principal component scores do exactly this, but these rely on X or XX^T, which we don't have here. We can only use the distances Δ_{ij}

Is this possible?

Unidentifiability



Distances Δ_{ij} are invariant under any orthonormal transformation $O \in \mathbb{R}^{p \times p}$ of $x_1, \ldots, x_n$ (i.e., $O^T O = I$)

Multidimensional scaling

Assume that $X \in \mathbb{R}^{n \times p}$ has been column centered (this is enough to deal with translation unidentifiability)

Multidimensional scaling (MDS): given distance matrix $\Delta \in \mathbb{R}^{n \times n}$, we

1. Recover the inner-product matrix $B = XX^T \in \mathbb{R}^{n \times n}$
2. Factorize B to get the first k principal component scores

(Called classical multidimensional scaling, there are other flavors, e.g., least squares multidimensional scaling)

We've already seen how to do step 2: remember that we compute the eigendecomposition $B = UD^2U^T$ and then the first k principal component scores are the first k columns of UD

So, how do we do step 1, i.e., how do we recover B?

Recovering inner-products from distances

The following procedure can be used to recover the inner-products $B = XX^T$ from Δ:

1. Compute $A_{ij} = -\frac{1}{2}\Delta_{ij}^2$ to form the matrix $A \in \mathbb{R}^{n \times n}$

2. Double center A—i.e., center both the columns and rows of A—to recover the matrix $B \in \mathbb{R}^{n \times n}$. Note that this is the same as:

$$B = (I - M)A(I - M)$$

where $M = \frac{1}{n}\mathbf{1}\mathbf{1}^T \in \mathbb{R}^{n \times n}$

$$M = \begin{pmatrix} \frac{1}{n} & \cdots & \frac{1}{n} \\ \vdots & & \vdots \\ \frac{1}{n} & \cdots & \frac{1}{n} \end{pmatrix}$$

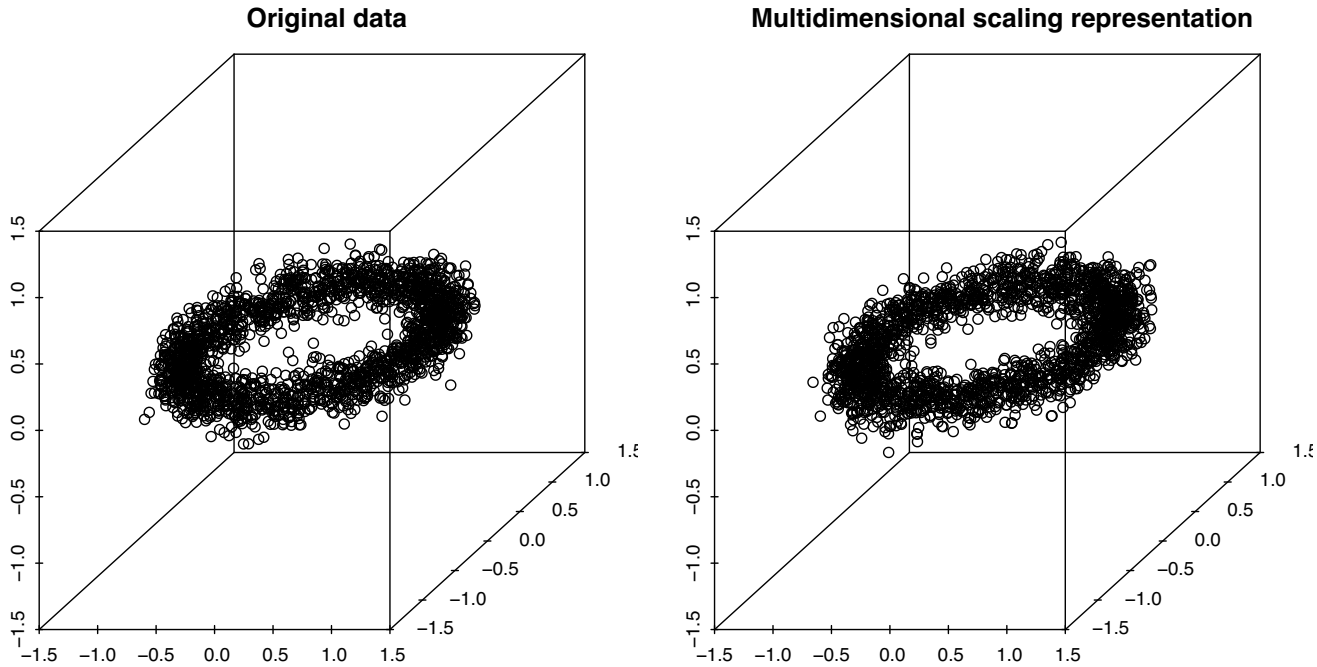
$$I - M = \begin{pmatrix} 1-\frac{1}{n} & -\frac{1}{n} & \cdots & -\frac{1}{n} \\ & 1-\frac{1}{n} & & \\ & & \ddots & \\ & & & 1-\frac{1}{n} \end{pmatrix} \quad (n \times n)$$

$$\left((I-M)A\right)_{11} = (1-\tfrac{1}{n})A_{11} - \tfrac{1}{n}A_{21} - \cdots$$

$$= A_{11} - \frac{1}{n}\sum A_{i1}$$

Example: donut data

Recall donut example: $X \in \mathbb{R}^{2000 \times 3}$. The left plot shows X and the right plot shows the multidimensional scaling representation $Z \in \mathbb{R}^{2000 \times 3}$ computed from the distance matrix $\Delta \in \mathbb{R}^{2000 \times 2000}$



The multidimensional scaling recovery is the same as the principal component scores $UD \in \mathbb{R}^{2000 \times 3}$. This is the same as X up to an orthonormal transformation—recall that $UD = XV$

[R example]

Beyond Euclidean distance

If the Δ_{ij} were actual Euclidean distances between the rows of a centered matrix $X \in \mathbb{R}^{n \times p}$, we get back the first k principal component scores exactly.

MDS would be just like doing PCA, except the result would not be identified up to an orthogonal transformation. Why do it?

Importantly, multidimensional scaling can be applied to any Δ_{ij}, not just Euclidean distances.

In these settings, MDS finds a set of points $Z \in \mathbb{R}^{n \times k}$ so that the distances between rows ~~$x_i, x_{i'}$~~ $z_i, z_{i'}$ are close to $\Delta_{ii'}$.

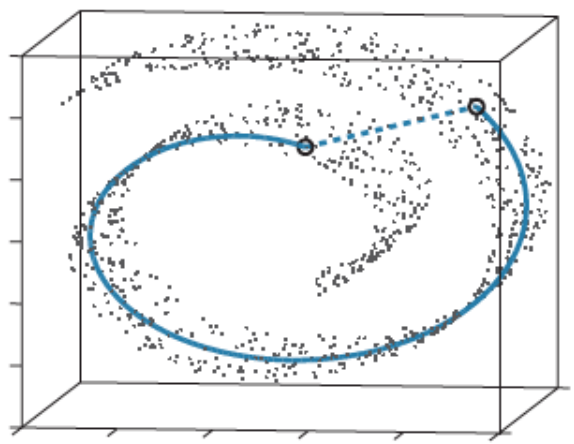
→ kernel PCA

Beyond Euclidean distance

There is a class of methods that construct a fancier metric Δ_{ij} between high-dimensional points $x_1, \ldots, x_n \in \mathbb{R}^p$, and then they feed these Δ_{ij} through multidimensional scaling to get a low-dimensional representation $z_1, \ldots, z_n \in \mathbb{R}^k$.

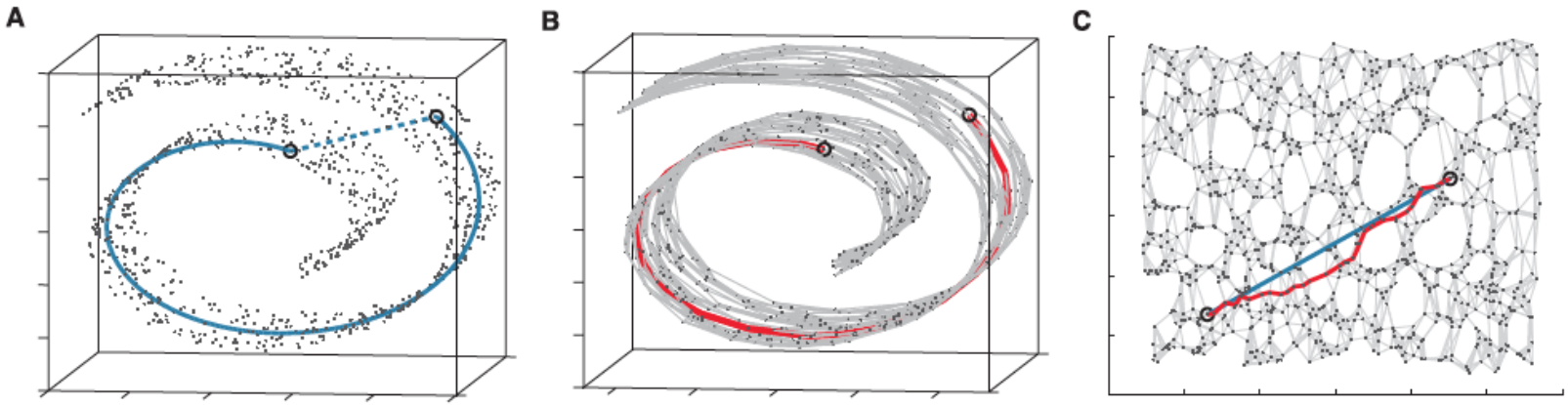
In this case, we don't just get principal component scores, and our low-dimensional representation can end up being a nonlinear function of the data

Why would we want to use non-Euclidean distances?



Isometric feature mapping

Isometric feature mapping[1] (Isomap) learns structure in a more general setting to define distances. The basic idea is to construct a graph $G = (V, E)$, i.e., construct edges E between vertices $V = \{1, \ldots, n\}$, based on the structure between $x_1, \ldots, x_n \in \mathbb{R}^p$. Then we define a graph distance $\Delta_{ij}^{\text{Isomap}}$ between i and j, and use multidimensional scaling for our low-dimensional representation



(From Tenenbaum et al. (2000))

[1] Tenenbaum et al. (2000), "A global geometric framework for nonlinear dimensionality reduction"

The Isomap graph distances

Constructing the graph: for each pair i, j, we connect i, j with an edge if either:

- x_i is one of x_j's m nearest neighbors, or
- x_j is one of x_i's m nearest neighbors

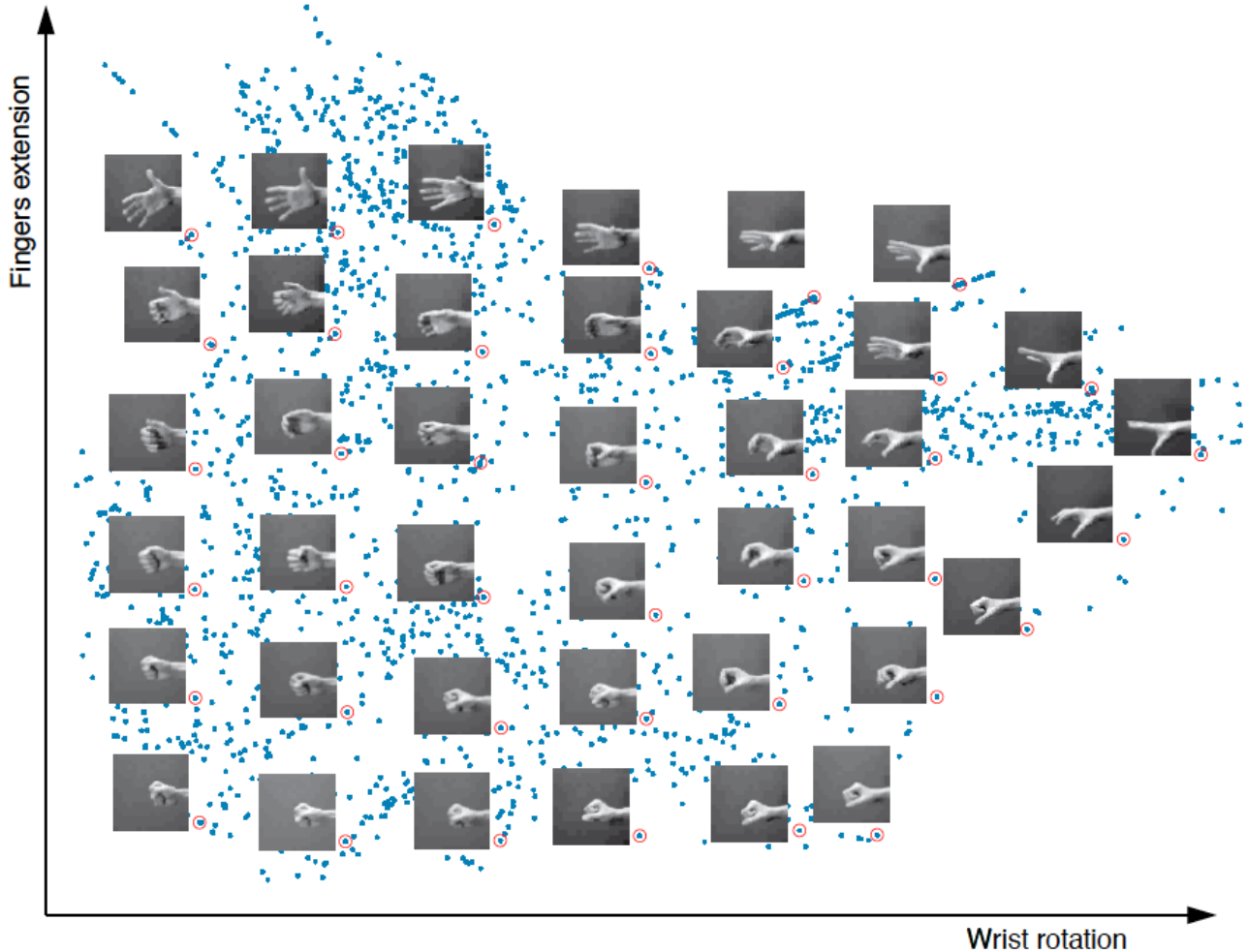
The weight of this edge $e = \{i, j\}$ is then $w_e = \|x_i - x_j\|_2$

Defining graph distances: now that we have built a graph, i.e., we have built an edge set E, we define the graph distance $\Delta_{ij}^{\text{Isomap}}$ between x_i and x_j to be the shortest path in our graph from i to j:

$$\Delta_{ij}^{\text{Isomap}} = \min_{\substack{\text{paths } P \subseteq E \\ \text{from } i \text{ to } j}} \sum_{e \in P} w_e$$

Example: hand positions

Example: $n = 3000$ images of hands and $p = 4096$, i.e., each image is 64 $\times$ 64 pixels. Isomap applied with $m = 6$ nearest neighbors, reduced to $k = 2$ dimensions:



(From `http://isomap.stanford.edu/handfig.html`)

Back to the SVD

$$\underset{n \times p}{X} = \underset{n \times p}{U} \underset{p \times p}{D} \underset{p \times p}{V^T}$$

Here $D = \mathrm{diag}(d_1, \ldots, d_p)$ is diagonal with $d_1 \geq \cdots \geq d_p \geq 0$, and U, V both have orthonormal columns. This gives us everything:

- columns of V, $v_1, \ldots, v_p \in \mathbb{R}^p$, are the principal component directions
- columns of U, $u_1, \ldots, u_p \in \mathbb{R}^n$, are the normalized principal component scores
- squaring the jth diagonal element of D and dividing by n, d_j^2/n, gives the variance explained by v_j

How do we think about the SVD?

X n users, p movies, ratings

$$X = UDV^T = \sum_{k=1}^{p} d_k u_k v_k^T$$

$u_k \in \mathbb{R}^n$, $v_k \in \mathbb{R}^p$, d_k scalar

u_k = (— users —)

v_k = (— movies —)

$u_k v_k^T = n \begin{pmatrix} & & \\ & o_{ij} & \\ & & \\ & p & \end{pmatrix}$ $(u_k)_i (v_k)_j$

[Example: SVD of time series]

v_1^T = (funniness of each movie)

u_1^T = (user's preference for funniness)

$$X = U D V^T$$

---

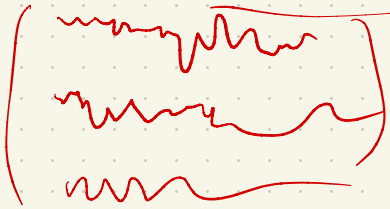
n assets

p time points

$$X = UDV^T = \sum_{k=1}^{p} d_k u_k v_k^T$$

$v_k^T \in \mathbb{R}^p$

$u_k \in \mathbb{R}^n$

V =  — time series

Matrix factorization and the SVD

We saw that one interpretation of the SVD could be as decomposing X into two matrices

$$X = WH = \sum_{k=1}^{K} w_k h_k^T$$

with $X \in \mathbb{R}^{n \times p}$, $W \in \mathbb{R}^{n \times K}$, and $H \in \mathbb{R}^{K \times p}$.

In this decomposition, X, W, and H are rank K. If $n > p$ and X is full column rank, then $K = p$.

However, we can *truncate* the SVD, just keeping $K < p$ terms, to get a simplified version of X.

Matrix factorization, Matrix reconstruction

We can think of this reduced-rank factorization problem as another optimization problem:

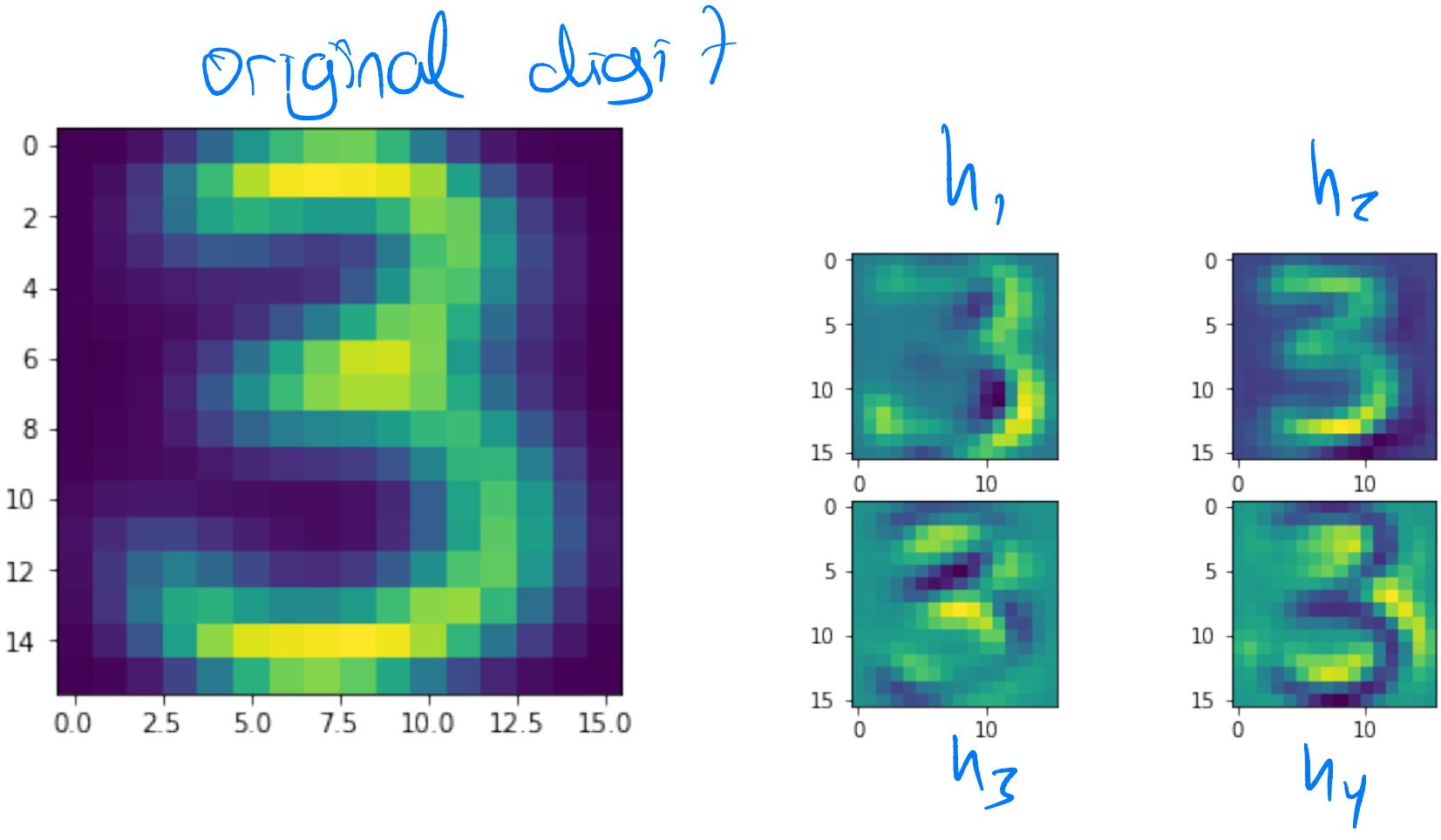
$$\min_{W,H} \|X - WH\|_F^2 = \min \sum_{i,j} (X_{ij} - (WH)_{ij})^2$$

This tries to approximate X as well as possible while restricting the number of allowed parameters.

This turns out to be equivalent to our reconstruction error recasting of PCA. Taking the first K components of the SVD solves this minimization problem!

(This idea has connections to the autoencoders we will see soon.)

PCA with digits



Once we see this problem posed as a simple optimization problem, many questions start to arise, leading to new methods.

- Do I want to put structure on W and/or H?
- Is Frobenius norm (squared error) the right distance measure?
- Can this idea be incorporated into probabilistic models?

Sparsity in W or H

$$X \approx WH = \sum_{k=1}^{K} w_k h_k^T$$

One common type of structure is sparsity. You might want to make:

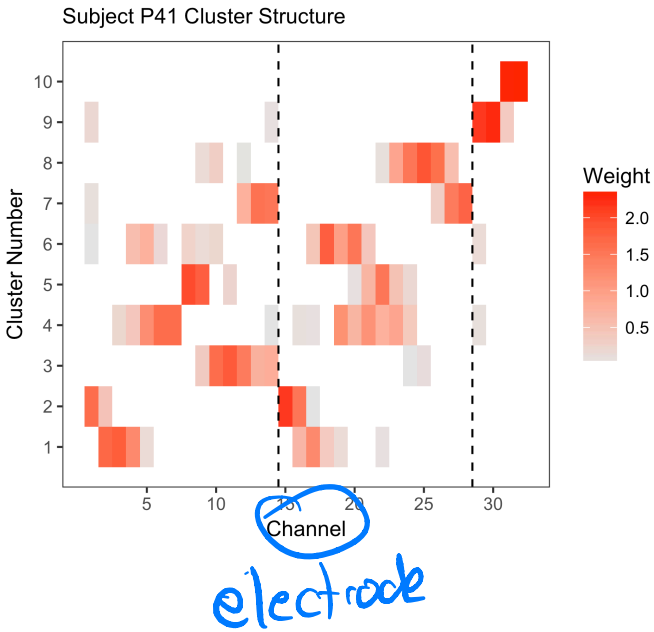
- The vectors w_k sparse. This would mean that each factor only applies to a subset of your observations. It gives a sense of overlapping clustering!

- The vectors h_k sparse. This would mean that each factor only uses a subset of your variables!
  - In the 3s example, each factor only involves a small number of pixels
  - In the time series example, each factor only involves some times

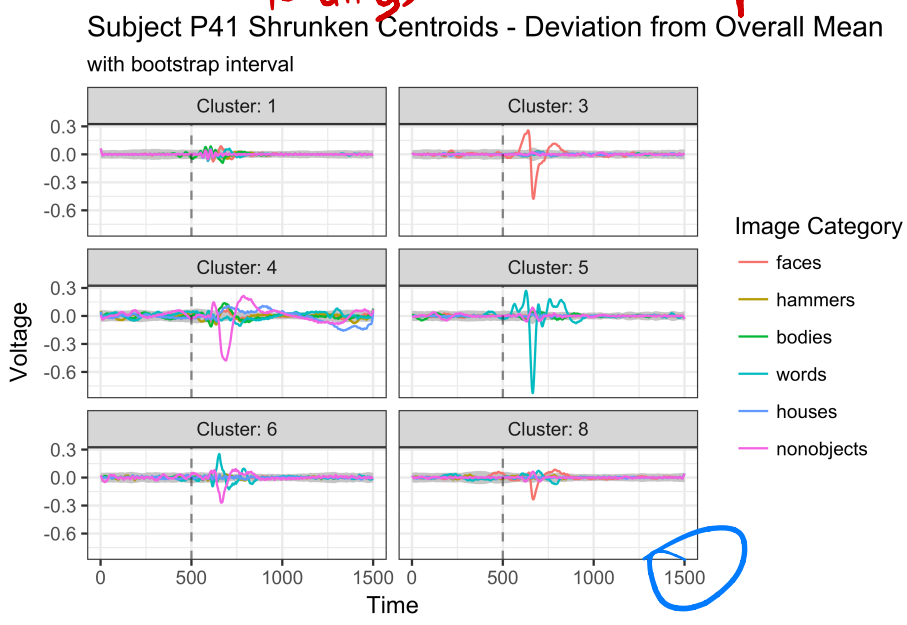
Sparse factorization from neuroscience

X = 30 rows (electrodes), 1500 columns (time) = W H

W: loadings; H: template time series



electrode



Non-negative Matrix Factorization (NMF)

Another approach is to decompose a matrix $X = WH$, but to force $W_{ij} \geq 0$, $H_{ij} \geq 0$.

$X \geq 0$

$$\min_{W,H} \|X - WH\|_F^2 \text{ such that } W \geq 0, H \geq 0$$

no orthogonality constraint on H!

This can be interpreted as a "source separation" sort of problem, with W_{ij} specifying how much of piece $H_{\cdot j}$ makes up observation $X_{i\cdot}$.

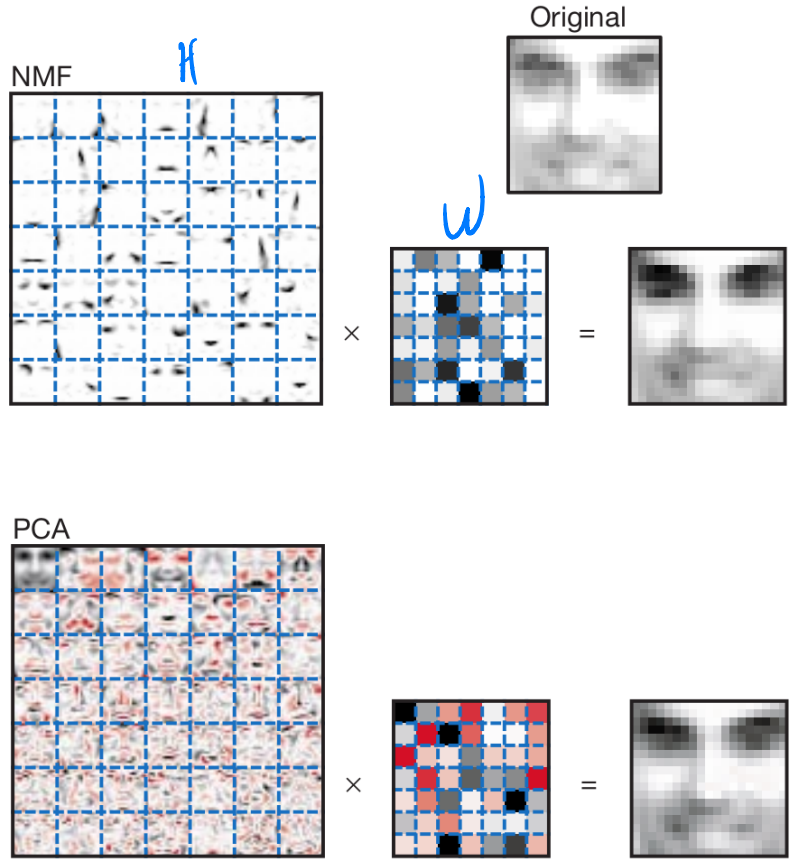
row i of X

column j of H

The non-negativity constraint automatically leads to sparsity in W and H.

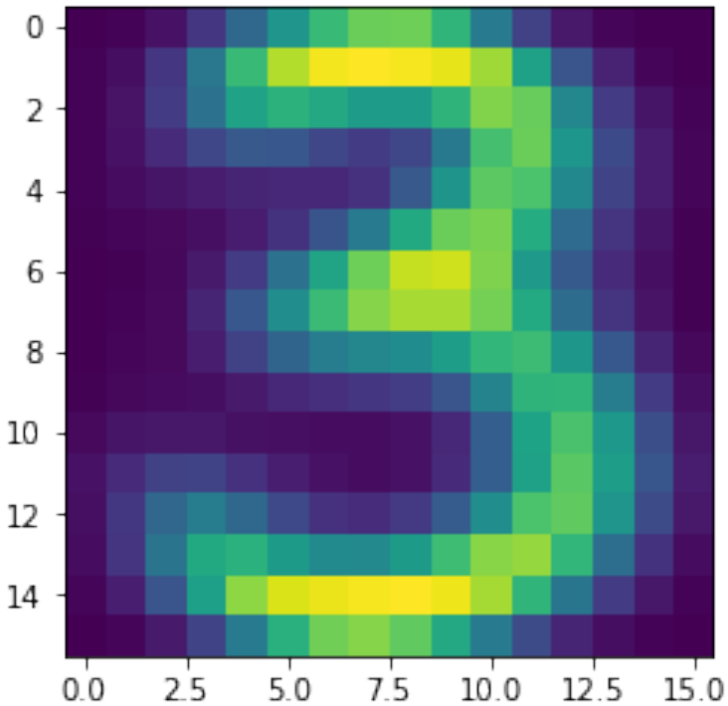
The Frobenius norm can be replaced with other loss functions, allowing this to be embedded in many models: for example, Poisson or Binomial.

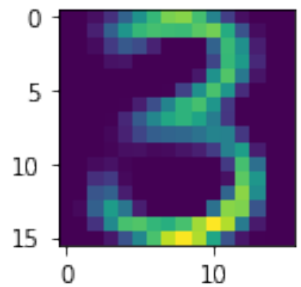
Non-negative Matrix Factorization (NMF)

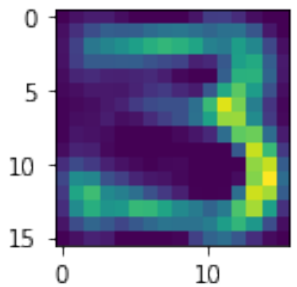


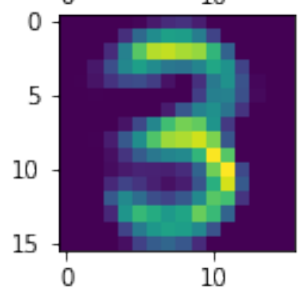
(Lee and Seung, 1999)

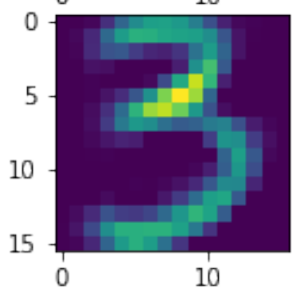
NMF with threes



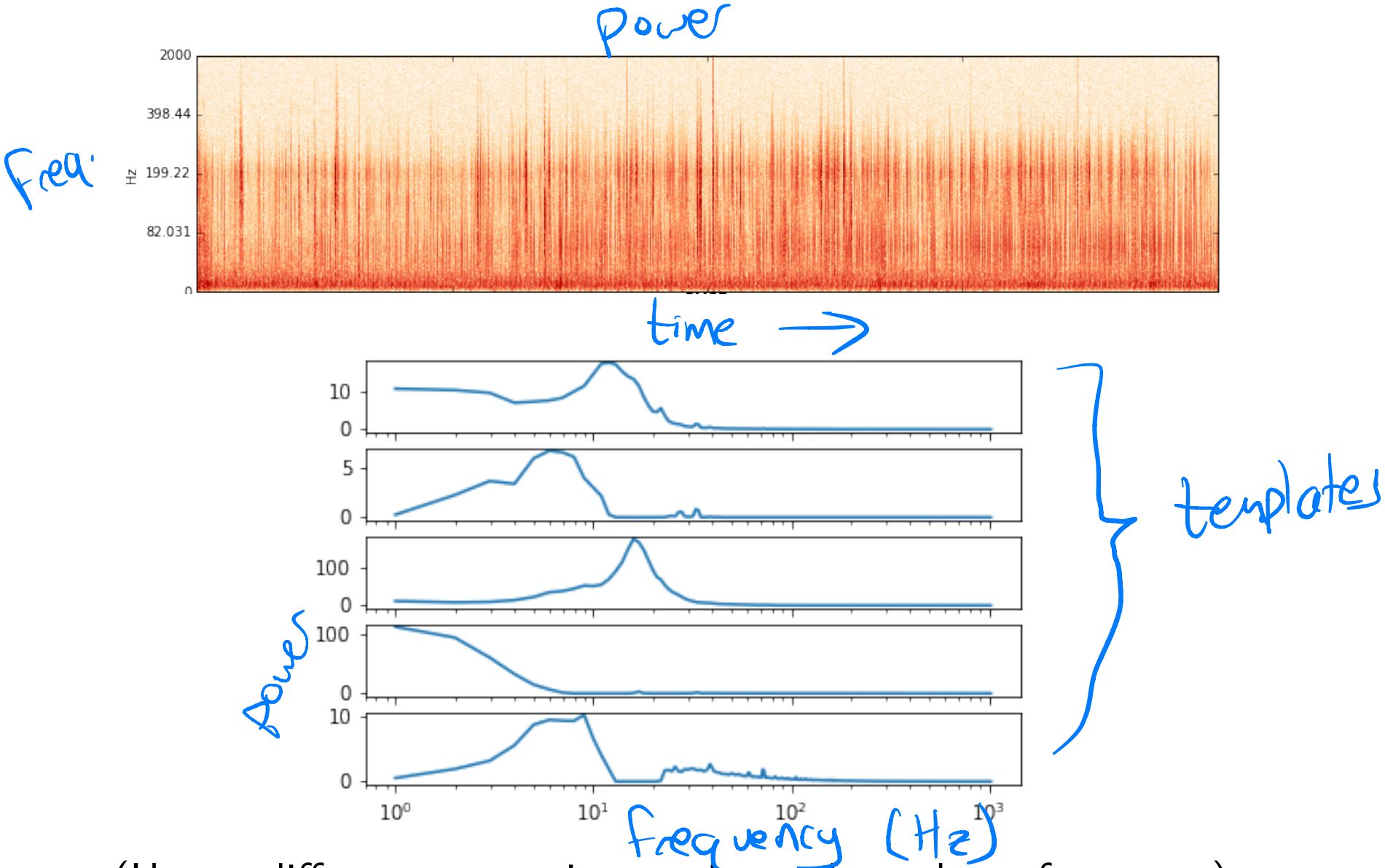








NMF with audio



(Uses a different norm, since variance depends on frequency)

Matrix Completion

Related to matrix factorization. Suppose that I have an X that I know to be low rank. If I don't observe some of the entries, I should still be able to guess what they are!

predicted — observed entries

$$\underset{X}{\text{minimize}}\, \text{rank}(X) \text{ such that } X_{ij} = M_{ij} \text{ for } i,j \in E$$

observed indices

This is a non-convex problem to solve exactly, but there are many approaches using convex relaxations or alternating minimization schemes.

Fancy combinations!

These decompositions can even be combined with other things you know. Robust PCA (Candes et. al., 2009) combines low-rank decomposition with sparse deviations:

$$\begin{aligned} \text{minimize } & \|L\|_* + \lambda \|S\|_1 \\ \text{subject to } & L + S = M \end{aligned}$$

This gives a low rank approximation L, which is able to fail in sparse regions

Robust PCA

$X = \begin{pmatrix} & \\ & \\ & \end{pmatrix}$ Pixels / Frames

(a) Original frames (b) Low-rank $\hat{L}$ (c) Sparse $\hat{S}$

(Candes et. al., 2009)

Factor Analysis

Very similar idea, but with a noise model on top of it. We could model

$$X - \mu = L F + \varepsilon$$

(L: weights; F: factors; ε: noise — model)

$$\mathrm{cov}(X) = \mathrm{cov}(\text{factors}) + \mathrm{cov}(\text{noise})$$

There are many such approaches. We can see how they would differ, since PCA would decompose the actual factor structure *plus the noise covariance structure*. However, this problem is non-convex.

Canonical Correlation Analysis

Worth mentioning briefly. Somewhat connected to these ideas, but doesn't fit anywhere else.

Imagine that I had two different sets of variables: $X \in \mathbb{R}^{n \times p}$ and $Z \in \mathbb{R}^{n \times q}$.

I might want to find combinations of X that are strongly correlated with combinations of Z.

That is:

$$\max_{\beta, \gamma} \mathrm{Corr}(X\beta, Z\gamma)$$

Canonical Correlation Analysis

$$\max_{\beta,\gamma} \mathrm{Corr}(X\beta, Z\gamma)$$

This is called Canonical Correlation Analysis (CCA). The solution looks similar. The best β is the first eigenvector of

$$\Sigma_{XX}^{-1/2}\Sigma_{XZ}\Sigma_{ZZ}^{-1}\Sigma_{ZX}\Sigma_{XX}^{-1/2}$$

Again, can add sparsity to β, γ to find small groups of X variables with high correlation with small groups of Z variables.

Mixture models

Suppose that we want to describe a distribution $P(X = x)$ as a *sum* of other known distributions:

$$f(x) = \sum_{k=1}^{K} \pi_k f_k(x)$$

where $\pi_k > 0$, $\sum_{k=1}^{K} \pi_k = 1$, and each f_k may have different parameters that must be estimated.

What does this mean generatively? To make new data,

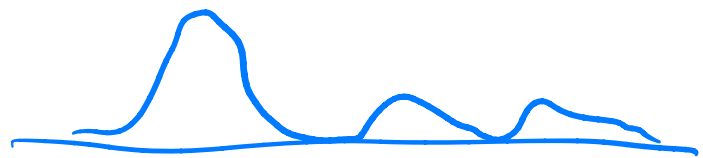
1. Pick the distribution based on π

2. Draw from $f_k(x)$

Mixture model applications

Why would we do this?

- Better than one parametric distribution, more interpretable than nonparametric distribution
- Because it's a good *generative* model for the data
- Processes that behave differently at different times (e.g. risk or return distributions) or are zero-inflated
- A form of parametric *soft* clustering
- Flexibly modeling multimodal or unusual distributions



How is this related to, say, QDA?

Mixture models are weird

Mixture model problems are often not well-posed.

Consider $K = 2$, and each f_k is just a normal density with separate μ_k and σ_k^2. We have iid data $x_1, \ldots, x_n$ drawn from the mixture, and want to estimate its parameters.

The model can be fit by maximum likelihood. The maximum likelihood is ∞, when $\mu_1 = x_k$, $\sigma_1 \to 0$, and $\sigma_2 > 0$. Oops.

How can we avoid this?

Identifiability

Mixture models are susceptible to label switching.

For example, if we have a mixture of five Normal distributions, we can shuffle π_k, μ_k, and σ_k and get an exactly equivalent model. There are $K!$ equivalent shuffles.

This means the likelihood is multimodal. Some estimation strategies struggle to deal with this, like MCMC.

Fitting a mixture model

To fit a mixture model, we must first decide

- The number of mixture components, K
- The form of each component (normal, gamma, t, Cauchy...)
- Whether any component parameters are fixed or constrained
- How to estimate the parameters

The MLE for a mixture model

all the parameters

The log-likelihood $\ell(\theta)$ of a mixture model is the log of the *sum* of the likelihoods $L_k(\theta)$ of the component mixtures. If we have data $x_1, \ldots, x_n$, then

$$\ell(\theta) = \sum_{i=1}^{N} \log \left(\sum_{k=1}^{K} \pi_k L_k(x_i) \right).$$

In general, the maximum has no analytical solution. We need numerical methods.

Expectation maximization (EM)

Return to the generative model. Suppose we also know $z_1, \ldots, z_n$, where $z_i = k$ if data point i was drawn from mixture component k. Then we have

$$\begin{aligned}
\ell_c(\theta) &= \sum_{i=1}^{n} \log \left(\sum_{k=1}^{K} \mathbb{I}(z_i = k) L_k(x_i) \right) \\
&= \sum_{i=1}^{n} \sum_{k=1}^{K} \mathbb{I}(z_i = k) \ell_k(x_i) \\
&= \sum_{i=1}^{n} \ell_{z_i}(x_i)
\end{aligned}$$

That's easy to maximize analytically. But how do we get $z_1, \ldots, z_n$?

Does this remind you of k-means?

The Expectation step

over dist. of $z|x$, parameters

Instead of $\ell_c(\theta)$, calculate $\mathbb{E}[\ell_c(\theta)]$:

$$\mathbb{E}[\ell_c(\theta)] = \sum_{i=1}^{n}\sum_{k=1}^{K} \mathbb{E}[\mathbb{I}(z_i = k)]\ell_k(x_i)$$

$$= \sum_{i=1}^{n}\sum_{k=1}^{K} \mathbb{P}(z_i = k)\ell_k(x_i).$$

The probabilities $\mathbb{P}(z_i = k)$ are called the responsibilities. They are easy to calculate.

$$P(z_i = k) = \frac{\pi_k f_k(x_i)}{\sum_{k=1}^{K} f_k(x_i)\pi_k}$$

The Maximization step

Once we have calculated $\mathbb{P}(z_i = k)$, plug the numbers in and maximize the $\mathbb{E}[\ell_c(\theta)]$ analytically:

$$\mathbb{E}[\ell_c(\theta)] = \sum_{i=1}^{n} \sum_{k=1}^{K} \mathbb{P}(z_i = k) \ell_k(x_i)$$

The overall EM procedure

1. Choose initial guesses for all parameters, including π_is.
2. Perform an E step and calculate the responsibilities. "new clusters"
3. Perform an M step and get new parameter estimates that maximize $\mathbb{E}[\ell_c(\theta)]$. "new centers"
4. Repeat 2 and 3 until parameters stop changing.

We can prove each step increases the likelihood $\ell(\theta)$, and we converge to a maximum.

EM is useful for more than just mixture models: it's useful for any model where possessing missing data would make the problem easier